

Class Test

1. What are the purposes of operating system? Describe the components of 2+4 operating system with proper figure.

Purposes of an Operating System:

1. **Resource Management:** Manages CPU, memory, I/O devices efficiently.
2. **Program Execution:** Loads and executes programs.
3. **File System Management:** Provides a way to store, organize, and access data.
4. **User Interface:** Provides CLI or GUI for user interaction.
5. **Security and Protection:** Protects data and resources from unauthorized access.

Components of Operating System:

Computer Hardware

- The physical components: CPU, memory, I/O devices.
- Provides the basic resources for the OS and applications.

Operating System (OS)

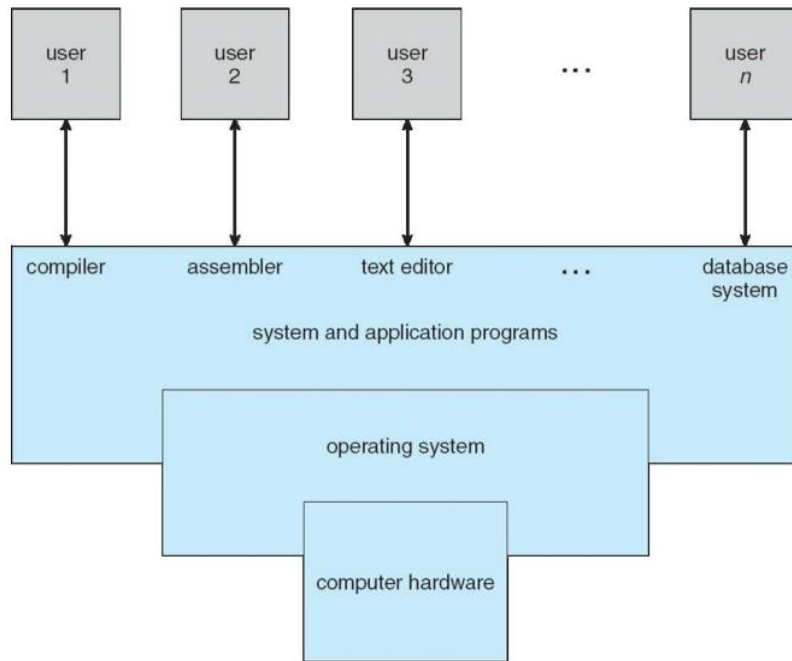
- Acts as an intermediary between a user of a computer and the computer hardware.
- Manages resources, provides services to system and application programs.
- Acts as an interface between hardware and users.

System and Application Programs

- Examples: compilers, assemblers, text editors, database systems.
- These programs use OS services to interact with hardware.

Users

- Interact with the system via applications or directly through the OS (CLI/GUI).
- Multiple users can interact simultaneously in a multi-user system.
- People, machines, other computers



2. What is bootstrap program? What does this bootstrap program do? 1+2

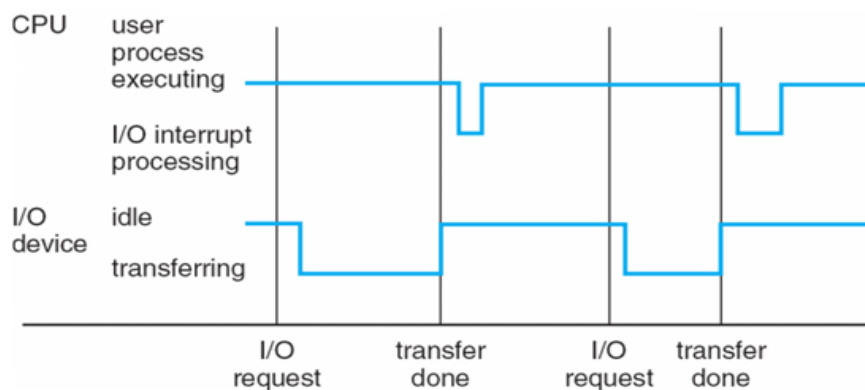
Definition:

- A **bootstrap program** is the first program that runs when we turn on or restart a computer.

Purpose / Functions:

1. It is stored in ROM or EPROM → This is permanent memory (called firmware).
2. It starts the computer and sets up everything (like memory, CPU, and devices).
3. It loads the operating system kernel into memory.
4. Then it starts the OS, and the computer is ready to use.

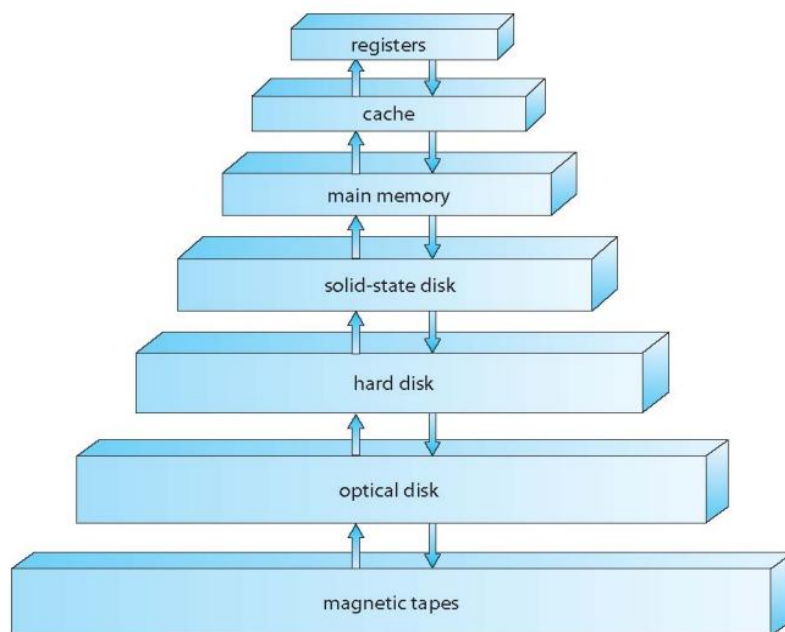
3. Briefly explain the interrupt timeline with I/O structure. 4



The interrupt timeline with I/O structure illustrates the interaction between a CPU and an I/O device during data transfer. Initially, the CPU executes a user process. When an I/O request is

made, the CPU initiates the transfer, and the I/O device begins transferring data while the CPU handles I/O interrupt processing. While the device is busy, the CPU often waits in a loop, checking if the I/O is finished. This waiting wastes CPU time because it cannot do other work. Also, only one I/O request can happen at a time—there is no multitasking for I/O. Once the transfer is done, the CPU resumes user process execution. This cycle repeats with each new I/O request and completion, showing how the CPU alternates between processing and handling interrupts to manage I/O operations efficiently.

4. Draw storage device hierarchy. 2



Explain operating system services with examples. 5

Operating System Services

1. User Interface (UI)

Most operating systems provide a user interface to interact with the system. This can vary in type:

- **Command-Line Interface (CLI):** Users type commands to perform operations.
- **Graphical User Interface (GUI):** Users interact with graphical elements like windows, icons, and menus.
- **Batch Interface:** Commands or programs are executed in batches without user interaction.

2. Program Execution

The operating system is responsible for loading programs into memory and executing them. It ensures that programs can:

- Start and run correctly.

- Terminate normally or handle abnormal termination by reporting errors.
3. **Input/Output (I/O) Operations**
 Programs often need to read input or produce output. The OS manages these operations from various sources, such as:
- **Files:** Reading from or writing data to files.
 - **Devices:** Handling input/output from hardware devices like keyboards, mice, printers, etc.
4. **File-System Manipulation**
 The operating system provides services to manage files and directories. Programs can:
- Create, read, write, search, and delete files or folders.
 - Manage file permissions to control access.
5. **Communications**
 Programs may need to exchange data either on the same system or over a network. The OS facilitates this through:
- **Shared memory:** Allowing multiple programs to access the same memory region.
 - **Message passing:** Enabling programs to send and receive messages.
6. **Error Detection**
 The OS continuously monitors for errors in both hardware and software, such as:
- Hardware errors in memory or I/O devices.
 - Errors in user programs.
- When an error is detected, the OS takes appropriate actions to correct the issue or notify the user, helping prevent system crashes or data loss. Good error-handling and debugging tools assist users and programmers in identifying and resolving problems quickly.

6. Describe Banker's Algorithm

Banker's Algorithm (Deadlock Avoidance)

The **Banker's Algorithm** is used in operating systems to **avoid deadlocks** by ensuring safe resource allocation. It works like a banker lending money: resources are allocated only if all processes can still finish safely.

Key Points:

1. **Concept:**
 - Each process declares its maximum resource requirement in advance.
 - The OS checks resource requests to ensure the system remains in a **safe state**.

2. Steps:

- **Check Request:** Ensure request $\leq$ Need and request $\leq$ Available.
- **Provisional Allocation:** Temporarily allocate requested resources.
- **Safety Check:** Simulate system to see if all processes can finish.
- **Grant or Delay:**
 - If safe $\rightarrow$ allocate resources.
 - If unsafe $\rightarrow$ delay request until resources are available.

3. Terminology:

- **Available:** Resources currently free.
- **Max:** Maximum resources a process may need.
- **Allocation:** Resources currently assigned.
- **Need:** Remaining resources needed to finish = Max – Allocation.

4. Advantage: Prevents deadlocks.

5. Disadvantage: Works only if maximum requirements are known; safety checks add overhead.

6. Mention the cpu scheduling criteria and describe with example

CPU Scheduling Criteria

CPU scheduling is done to **decide which process gets the CPU next**. The effectiveness of scheduling is measured using several **criteria**:

1. CPU Utilization

- Keep the CPU **as busy as possible**.
- **Example:** If CPU is idle 10% of the time, utilization is 90%. Higher is better.

2. Throughput

- Number of processes **completed per unit time**.
- **Example:** If 5 processes finish in 1 second, throughput = 5 processes/sec.

3. Turnaround Time

- Total time taken from **process submission to completion**.
- **Example:** A process submitted at time 0 and finished at 10 $\rightarrow$ turnaround time = 10 units.

4. Waiting Time

- Total time a process **waits in the ready queue** before getting CPU.
- **Example:** A process waits 2 units in queue, runs 3 units $\rightarrow$ waiting time = 2 units.

5. Response Time

- Time from **submission to first response/output**.
- Important for **interactive systems**.
- **Example:** A user request sent at time 0, first output at time 2 → response time = 2 units.

Questions

1.

a) Define interrupt in operating system. Differentiate between interrupt and polling. (03)

b) Explain Linux process tree with diagram. Describe how processes are created and terminated in Linux. (04)

c) Define trap in operating system. Explain the steps in making a system call with necessary diagrams. (05)

1a) Interrupt (3 Marks)

Definition:

An **interrupt** is a signal sent by hardware or software to the CPU to indicate that it needs **immediate attention**. When an interrupt occurs, the CPU **pauses the current process**, saves its state, and executes the interrupt handler.

Difference between Interrupt and Polling:

Aspect	Interrupt	Polling
Definition	CPU is notified by hardware/software when attention is needed	CPU repeatedly checks device status at intervals
CPU Utilization	Efficient; CPU does other work until interrupt occurs	Inefficient; CPU wastes cycles checking status
Response Time	Fast; immediate response to event	Slower; response depends on polling frequency
Control	Device controls CPU by sending interrupt	CPU controls the process by checking device

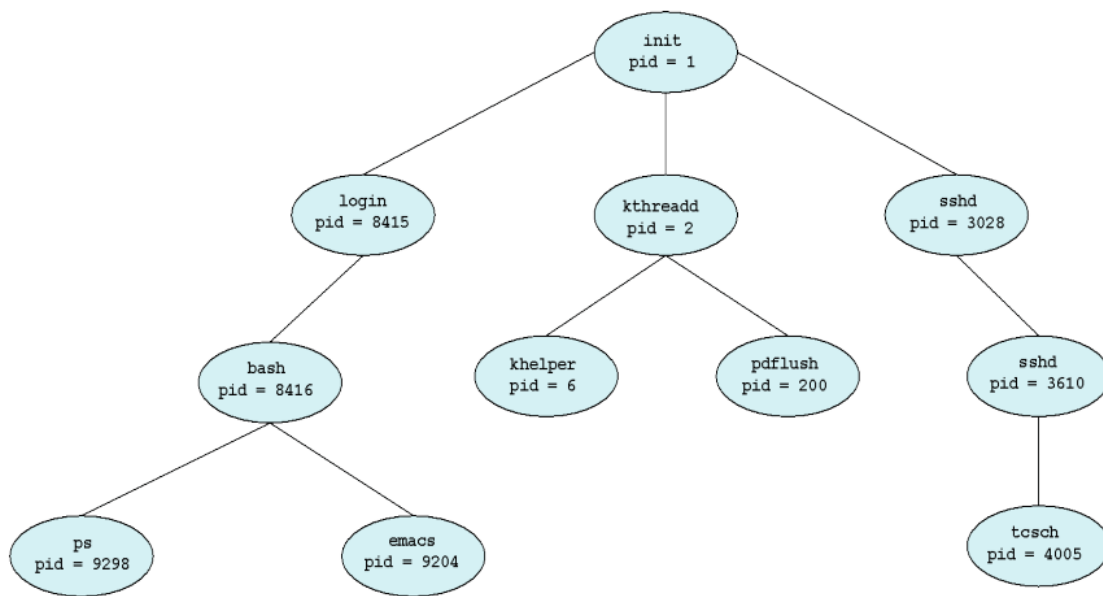
1b) Linux Process Tree (4 Marks)

Process Tree:

- Linux organizes processes in a **hierarchical tree**.
- Every process has a **Parent Process ID (PPID)**.
- The first process is **init (PID 1)**, which is ancestor of all other processes.

Polling is a CPU-driven technique where the processor keeps repeatedly checking (polling) a device's status register to see whether it needs attention (like if data is ready to be read/written).

Diagram-



Optional:

Here's a breakdown of the tree:

- The root process is init with PID 1, which is the first process started by the kernel and the parent of all other processes.
- init spawns three direct child processes:
 - login (PID 8415), which is a process for user login.
 - kthreadd (PID 2), a kernel thread that manages kernel threads.
 - sshd (PID 3028), a secure shell daemon that allows remote logins.
- login (PID 8415) spawns bash (PID 8416), a command-line shell.
- bash (PID 8416) spawns emacs (PID 9204), a text editor.
- kthreadd (PID 2) spawns khelper (PID 6), a helper kernel thread, and pdflush (PID 200), a process that writes dirty buffers to disk.
- sshd (PID 3028) spawns another sshd (PID 3610), which in turn spawns tcsch (PID 4005), likely a typo or shorthand for a process related to the shell or a specific task.

This tree illustrates how processes are created and managed, with each child process inheriting properties from its parent, and it reflects a snapshot of the system's process hierarchy at a given moment.

Process Creation:

- A process is created using the **fork() system call**, which **duplicates the parent process**.

- `exec()` can be used to **replace the process image** with a new program.

Process Termination:

- A process can terminate using `exit()`.
- The parent should **read the exit status** using `wait()` or `waitpid()` to prevent zombies.

1c) Trap (5 Marks)

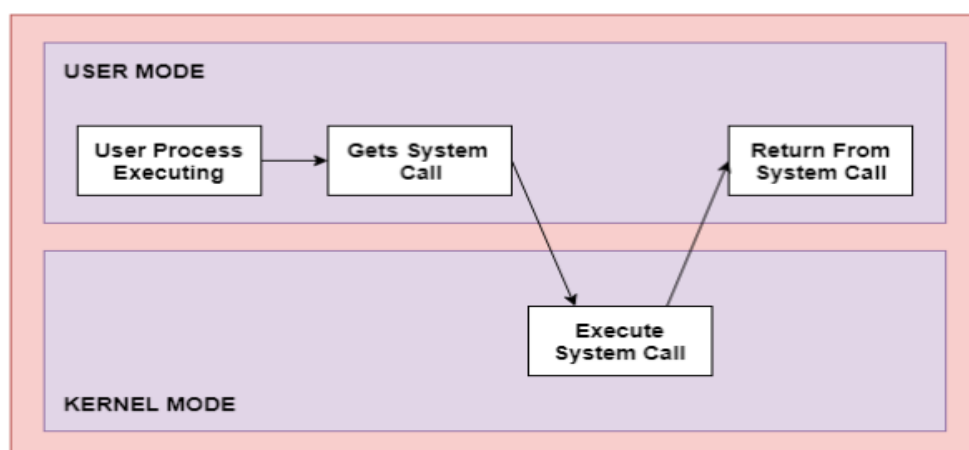
Definition:

A **trap** is a **synchronous software-generated interrupt** caused by an exceptional condition in a process, e.g., division by zero or invalid memory access.

Steps in Making a System Call

1. **User Request** – A user program requests an OS service (e.g., file read/write) using a library function.
2. **System Call Interface** – The library function passes the system call number and parameters to the OS.
3. **Trap Instruction** – A software interrupt (trap) switches the CPU from **user mode to kernel mode**.
4. **OS Handler** – The system call handler identifies the requested service.
5. **Service Execution** – The OS executes the corresponding service routine in **kernel mode**.
6. **Return to User** – Results are returned, CPU switches back to **user mode**, and control goes to the user program.

Diagram (System Call Flow):



2.

- a) Define symmetric and asymmetric multiprocessing in operating system. Differentiate between thread and process. (03)
- b) Write a short note on each of the following Linux commands: (04)

1. touch
2. chmod
3. cat
4. pwd

c) Define context switching. Explain all the process states in an operating system along with the process state diagram. (05)

2a) Multiprocessing and Threads vs Processes (3 Marks)

Symmetric Multiprocessing (SMP):

- All CPUs share **common memory and I/O devices**.
- Each CPU can execute **any process or thread** independently.
- Example: Modern multi-core systems.

Asymmetric Multiprocessing (AMP):

- One **master CPU** controls the system, others are **slaves**.
- Master assigns tasks to slave CPUs.
- Example: Some embedded systems.

Difference between Thread and Process:

Aspect	Process	Thread
Memory	Has separate memory space	Shares memory with other threads
Creation	Heavyweight; slower	Lightweight; faster
Communication	Needs IPC (pipes, message queue)	Can communicate via shared memory
Execution	Independent	Runs within a process

2b) Short Note on Linux Commands (4 Marks)

1. **touch** – Creates an empty file or updates the timestamp of an existing file.
touch file.txt
2. **chmod** – Changes the **permissions** of a file or directory.
chmod 755 file.sh
3. **cat** – Displays the **contents of a file** on the terminal or can concatenate multiple files.
cat file.txt
4. **pwd** – Prints the **current working directory** of the terminal.
pwd

2c) Context Switching and Process States

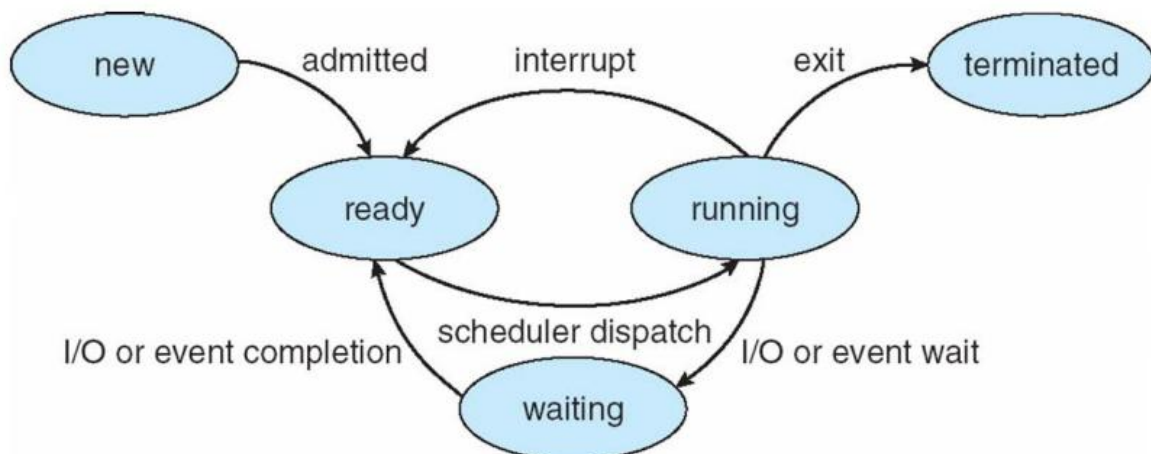
Context Switching:

- It is the **mechanism of saving the state** of a running process and loading the state of the next process to run.
- Ensures CPU can switch between multiple processes efficiently.

Process States in OS:

1. **New:** Process is being created.
2. **Ready:** Process is waiting in the **ready queue** for CPU allocation.
3. **Running:** Process is currently executing on the CPU.
4. **Waiting / Blocked:** Process is waiting for **I/O or an event** to complete.
5. **Terminated / Exit:** Process has finished execution and is removed from memory.

Process State Diagram:



Explanation:

- **New → Ready:** Process creation complete.
- **Ready → Running:** Scheduler allocates CPU.
- **Running → Waiting:** Process needs I/O or event.
- **Running → Ready:** Preemption by scheduler.
- **Running → Terminated:** Process finishes execution.
- **Waiting → Ready:** Event completed; process ready to run.

3.

a) Explain why zombie processes exist and how they are created. Describe how you can prevent

the creation of zombie processes. (03)

b) Differentiate between wait() and waitpid() system calls in Unix. Mention their necessity in terms of Unix kernel-based operating systems. (04)

c) Write a C program that creates a child process using fork(). The child process should print "Hello from child" and exit, while the parent process should wait for the child to finish and then print "Child has finished". (05)

3a) Zombie Processes (3 Marks)

Definition / Why They Exist:

- A **zombie process** is a process that has **finished execution but still has an entry in the process table**.
- They exist because the **parent process has not yet read the exit status** of the child using wait() or waitpid().

How They Are Created:

1. A child process finishes execution using exit().
2. Its **PCB (Process Control Block)** remains in the process table to store the exit status.
3. Until the parent collects the status, the child remains a zombie.

Prevention:

- Use wait() or waitpid() in the parent to collect the child's exit status.
- For orphaned children, init (PID 1) automatically adopts them and reaps their exit status.

3b) Difference between wait() and waitpid() (4 Marks)

Aspect	wait()	waitpid()
Purpose	Waits for any child process to finish	Waits for a specific child process
Blocking	Blocks until a child exits	Can be blocking or non-blocking (with options)
Return Value	Returns PID of the terminated child	Returns PID of the child specified in the call
Flexibility	Less flexible, no control over which child	More flexible, allows specifying child PID and options like WNOHANG

Necessity in Unix Kernel:

- Prevent **zombie processes** by reading child exit status.
- Allows **synchronization** between parent and child processes.
- waitpid() provides **better control** for multiple children in complex programs.

3c) C Program Using fork()

```
#include <bits/stdc++.h>
int main() {
    pid_t pid = fork(); // Create child process

    if (pid < 0) {
        // Fork failed
        printf("Fork failed\n");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("Hello from child\n");
        exit(0); // Child exits
    }
    else {
        // Parent process
        wait(NULL); // Wait for child to finish
        printf("Child has finished\n");
    }

    return 0;
}
```

Output Example:

Hello from child

Child has finished

4.

a) What is a signal in the Unix system? Mention at least six signals found in most Unix systems. (03)

b) Explain the three possible ways to handle a kernel-generated SIGCHLD signal by a parent process. (04)

c) A disk contains 200 tracks (0–199). The request queue contains track no. 82, 170, 43, 140, 24, 16, 190 respectively. Current position of R/W head is 50. Calculate the total number of track movement by R/W head using C-SCAN algorithm. (05)

5.

a) Define deadlock. Differentiate between livelock and starvation. (03)

b) Mention the four necessary conditions for occurring deadlock. Explain the methods for handling deadlock. (04)

c) A program has four processes (P0, P1, P2, P3) and four resource types (A, B, C, D). The following table shows the maximum need and currently allocated resources for each process:

Process	Allocation (A,B,C,D)	Max Need (A,B,C,D)
P0	0, 1, 0, 2	7, 5, 3, 3
P1	2, 0, 0, 0	3, 2, 2, 2
P2	3, 0, 0, 2	9, 0, 2, 2
P3	2, 1, 1, 1	2, 2, 2, 2

The total available resources are {3, 3, 2, 2}. Use the Banker's algorithm to determine if the system is in a safe state or not. If it is in a safe state, show the sequence in which the processes can complete safely. (05)

a) Deadlock, Livelock, and Starvation

Definition of Deadlock:

Deadlock is a situation in an operating system where two or more processes are waiting for resources in such a way that none of them can proceed further.

- Example: Process A waits for a resource held by Process B, and Process B waits for a resource held by Process A → both are stuck forever.

Difference between Livelock and Starvation:

Aspect	Livelock	Starvation
Definition	A situation where processes keep changing their states in response to each other but still cannot make progress.	A process waits indefinitely because resources are continuously given to other processes with higher priority.
Progress	Processes are active but doing no useful work.	Process is ready to run but never gets CPU or resource.
Cause	Usually caused by processes constantly reacting to each other (e.g., retrying).	Caused by unfair scheduling or priority-based allocation.
Example	Two people step aside repeatedly to let the other pass, but neither moves forward.	A low-priority process never executes because higher-priority processes always get CPU.

b) Necessary Conditions for Deadlock

- **Mutual Exclusion:** A resource (like a printer or file) can only be used by one process at a time.
- **Hold and Wait:** A process is holding at least one resource while also waiting to acquire additional resources that are currently being used by other processes.
- **No Preemption:** A process cannot be forced to release a resource; it must give it up voluntarily after finishing its use.

- **Circular Wait:** There exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for a resource held by P_2 , P_2 is waiting for a resource held by P_3 , ..., and P_n is waiting for a resource held by P_0 .

Methods for Handling Deadlock

1. Deadlock Prevention

- Ensure at least one of the four necessary conditions cannot hold.
- Example:
 - Remove **hold and wait** by forcing processes to request all resources at once.
 - Allow **preemption** of resources.

2. Deadlock Avoidance

- Dynamically examine resource allocation to avoid unsafe states.
- Example: **Banker's Algorithm** is used to ensure the system always stays in a safe state.

3. Deadlock Detection and Recovery

- Allow deadlock to occur but detect it using algorithms (e.g., wait-for graph).
- Recover by terminating processes or preempting resources.

4. Deadlock Ignorance

- Assume deadlocks are rare and let them be ignored.
- Used in many systems (e.g., UNIX, Linux, Windows) because handling is costly.

c) Given

- Allocation:
 - $P_0 = (0, 1, 0, 2)$
 - $P_1 = (2, 0, 0, 0)$
 - $P_2 = (3, 0, 0, 2)$
 - $P_3 = (2, 1, 1, 1)$
- Max Need:
 - $P_0 = (7, 5, 3, 3)$
 - $P_1 = (3, 2, 2, 2)$
 - $P_2 = (9, 0, 2, 2)$
 - $P_3 = (2, 2, 2, 2)$
- Available (initial) = $(3, 3, 2, 2)$

1) Compute Need = Max – Allocation

- $\text{Need}(P_0) = (7, 5, 3, 3) - (0, 1, 0, 2) = (7, 4, 3, 1)$

- $\text{Need}(P1) = (3,2,2,2) - (2,0,0,0) = \mathbf{(1,2,2,2)}$
- $\text{Need}(P2) = (9,0,2,2) - (3,0,0,2) = \mathbf{(6,0,2,0)}$
- $\text{Need}(P3) = (2,2,2,2) - (2,1,1,1) = \mathbf{(0,1,1,1)}$

2) Banker's safety check (find a sequence)

Start Available = **(3,3,2,2)**.

- Can P0 run? $\text{Need}(7,4,3,1) \leq \text{Avail}(3,3,2,2)$? **No** ($7 > 3$).
- Can P1 run? $\text{Need}(1,2,2,2) \leq (3,3,2,2)$? **Yes**.
→ Let P1 finish; release $\text{Allocation}(P1) = (2,0,0,0)$.
New Available = $(3,3,2,2) + (2,0,0,0) = \mathbf{(5,3,2,2)}$.
Safe sequence so far: **P1**
- With Avail = $(5,3,2,2)$:
P0? $\text{Need}(7,4,3,1) \leq (5,3,2,2)$? **No** ($7 > 5$).
P2? $\text{Need}(6,0,2,0) \leq (5,3,2,2)$? **No** ($6 > 5$).
P3? $\text{Need}(0,1,1,1) \leq (5,3,2,2)$? **Yes**.
→ Let P3 finish; release $\text{Allocation}(P3) = (2,1,1,1)$.
New Available = $(5,3,2,2) + (2,1,1,1) = \mathbf{(7,4,3,3)}$.
Safe sequence: **P1 → P3**
- With Avail = $(7,4,3,3)$:
P0? $\text{Need}(7,4,3,1) \leq (7,4,3,3)$? **Yes**.
→ Let P0 finish; release $\text{Allocation}(P0) = (0,1,0,2)$.
New Available = $(7,4,3,3) + (0,1,0,2) = \mathbf{(7,5,3,5)}$.
Safe sequence: **P1 → P3 → P0**
- With Avail = $(7,5,3,5)$:
P2? $\text{Need}(6,0,2,0) \leq (7,5,3,5)$? **Yes**.
→ P2 can finish.

Safe sequence completes: **P1 → P3 → P0 → P2**

6.

a) Explain the role of dispatcher and CPU scheduler in operating system. Explain whether they are the same with proper reasoning. (03)

b) Does the following instruction set create a deadlock? Explain with proper diagram. (04)

- Process 1 holds R and S.
- Process 2 holds nothing but wants T.
- Process 3 holds nothing but wants S.
- Process 4 holds U and wants S and T.
- Process 5 holds T and wants V.

- Process 6 holds W and wants S.
- Process 7 holds V and wants U.

c) Consider a system with the following processes, each with their arrival time (AT), burst time (BT), and quantum = 4:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	6
P2	1	4
P3	2	9
P4	3	5
P5	4	2

Using the Round Robin scheduling algorithm, compute the average waiting time and turnaround time for these processes. (05)

a) Dispatcher and CPU Scheduler

CPU Scheduler:

The CPU scheduler selects one process from the ready queue to execute next on the CPU. It uses scheduling algorithms (like FCFS, SJF, Priority, or Round Robin) to improve CPU utilization, throughput, and response time. It is also called the *short-term scheduler* because it makes frequent decisions.

Dispatcher:

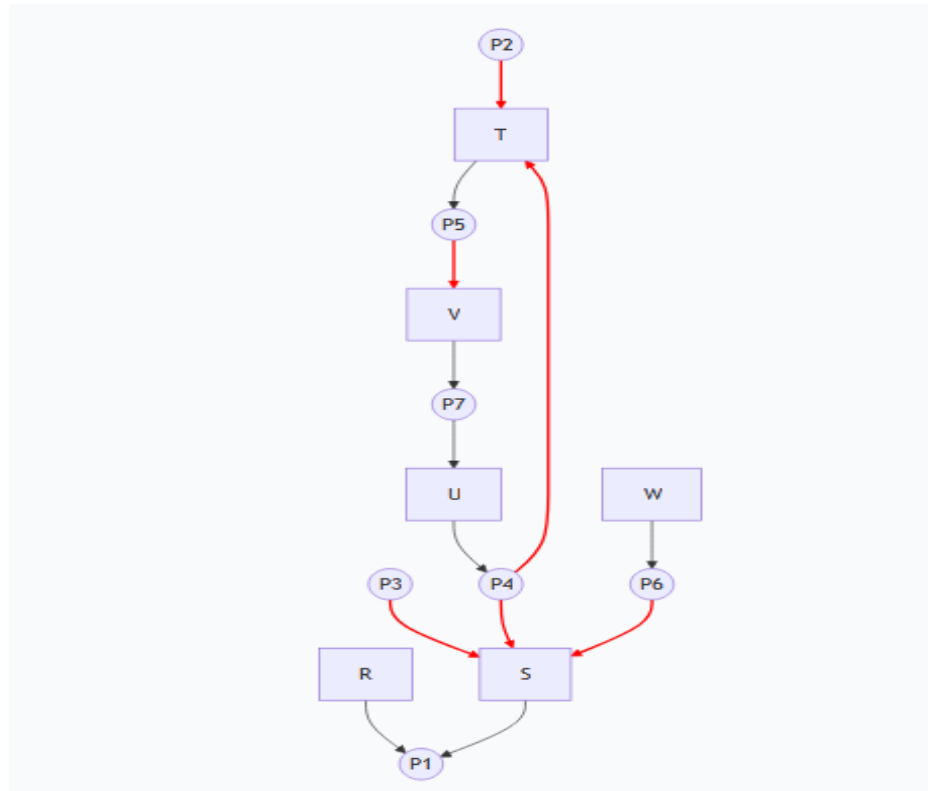
The dispatcher gives control of the CPU to the process chosen by the CPU scheduler. It performs context switching, changes the mode from kernel to user, and transfers execution to the selected process. Its speed is very important because too much dispatch latency reduces overall CPU performance.

Are they the same?

No, they are not the same. The CPU scheduler makes the decision *which process should run*, while the dispatcher performs the *actual switching*. They work together, but their roles are different.

b) Key:

- Solid arrows from resources to processes indicate allocation (e.g., S → P1 means resource S is allocated to process P1).
- Solid arrows from processes to resources indicate request (e.g., P2 → T means process P2 is requesting resource T).
- Red arrows highlight the deadlock cycle



Deadlock Cycle Explanation:

The graph contains a cycle:

1. **P4** holds resource U and is requesting resources S and T.
2. **P5** holds resource T and is requesting resource V.
3. **P7** holds resource V and is requesting resource U.

This forms a circular wait:

P4 → T (held by P5) → P5 → V (held by P7) → P7 → U (held by P4) → P4

Additional Blocked Processes:

- **P2** is requesting T (held by P5), so it is blocked by the deadlock.
- **P3** and **P6** are requesting S (held by P1). Since P1 is not part of any cycle and can complete, these processes are not deadlocked (they can proceed once P1 releases S).

c)

Given:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	6
P2	1	4

P3	2	9
P4	3	5
P5	4	2

Time Quantum = 4

Step 1: Gantt Chart (RR scheduling)

We will schedule processes in order of arrival, preempting after 4 units.

1. **P1** starts at 0 → runs 4 units → remaining BT = 6 – 4 = 2 → Time = 0–4
2. **P2** runs 4 units (BT = 4) → finishes → Time = 4–8
3. **P3** runs 4 units → remaining BT = 9 – 4 = 5 → Time = 8–12
4. **P4** runs 4 units → remaining BT = 5 – 4 = 1 → Time = 12–16
5. **P5** runs 2 units → finishes → Time = 16–18
6. Back to **P1** (remaining BT = 2) → runs 2 units → finishes → Time = 18–20
7. Back to **P3** (remaining BT = 5) → runs 4 units → remaining BT = 1 → Time = 20–24
8. Back to **P4** (remaining BT = 1) → runs 1 unit → finishes → Time = 24–25
9. Back to **P3** (remaining BT = 1) → runs 1 unit → finishes → Time = 25–26

Time	0	4	8	12	16	18	20	24	25	26
Process	P1	P2	P3	P4	P5	P1	P3	P4	P3	

Step 2: Completion Time (CT)

Process	Completion Time (CT)
P1	20
P2	8
P3	26
P4	25
P5	18

Step 3: Turnaround Time (TAT = CT – AT)

Process	CT	AT	TAT = CT – AT
---------	----	----	---------------

P1	20	0	20
P2	8	1	7
P3	26	2	24
P4	25	3	22
P5	18	4	14

Step 4: Waiting Time (WT = TAT – BT)

Process	TAT	BT	WT = TAT – BT
P1	20	6	14
P2	7	4	3
P3	24	9	15
P4	22	5	17
P5	14	2	12

Step 5: Average TAT and WT

- **Average Turnaround Time (TAT)** = $(20 + 7 + 24 + 22 + 14) / 5 = 17.4$
- **Average Waiting Time (WT)** = $(14 + 3 + 15 + 17 + 12) / 5 = 12.2$

7.

a) What do you understand by atomic instructions? Why are they necessary to solve the critical section problem?

b) Assume you have the following processes to execute with a single processor. The processes arrive at the times shown and have the given CPU burst times and priorities:

Process	Arrival Time	CPU Burst Time	Priority
A	0	8	3
B	3	4	1
C	5	7	4
D	8	3	2

For each of the following scheduling algorithms:

- Shortest Job First (SJF), Preemptive
- Priority Scheduling, Preemptive

1. Draw the Gantt chart.
2. Calculate the average waiting time.

3. Calculate the turnaround time of each process.

c) Write down the advantages and disadvantages of the Round Robin Scheduling algorithm.

a) Atomic Instructions

Atomic instructions are machine-level instructions that **execute completely or not at all**, without being interrupted by other processes. Examples include test-and-set, swap, and fetch-and-increment.

Importance in Critical Section Problem:

- In the critical section problem, multiple processes share resources, and **only one process should enter the critical section at a time**.
- Atomic instructions ensure operations like **checking and setting a lock** happen in **one uninterruptible step**.
- Without atomicity, **race conditions** may occur, allowing multiple processes to enter the critical section simultaneously.

Example: Using test-and-set ensures that a process can check and acquire a lock safely in one step, maintaining **mutual exclusion**

c) Advantages of Round Robin (RR) Scheduling

1. **Fairness:** Each process gets an equal share of CPU time in a cyclic order.
2. **Preemption:** Prevents any single process from monopolizing the CPU.
3. **Good for time-sharing systems:** Provides quick response for interactive processes.
4. **Predictable response time:** Each process will wait at most $(n-1) \times \text{quantum}$ before getting CPU again.

Disadvantages of Round Robin (RR) Scheduling

1. **Context Switching Overhead:** Frequent context switches can reduce CPU efficiency.
2. **Poor for long processes:** Processes with long burst times may take many cycles to complete.
3. **Quantum selection issue:**
 - If quantum is too small $\rightarrow$ too many context switches $\rightarrow$ high overhead.
 - If quantum is too large $\rightarrow$ behaves like FCFS $\rightarrow$ poor response time.
4. **Average waiting time may be high:** Especially if burst times vary significantly.

a) "Raid level 01 or 10 increase disk access performance and availability", discuss with a suitable example.

b) Determine the total head movement for algorithms SSTF and C-Look for the following requests (in track number): 98, 181, 23, 65, 122, 14, 72, 36, 26, 67; where the current position of head is on track number 0.

c) A disk has an average seek time of 5ms, a rotational speed of 15,000 rpm, and 500 sectors per track. What is the average time to read a single sector? What is the expected time to read 500 contiguous access time to the same track? What is the expected time to read 500 sectors scattered over the disk?